



Summarize Data with Pandas

DATA SCIENCE WITH PYTHON

Contents:

- `info()`
- `describe()`
- `value_counts()`
- `nunique()`
- `sum()`
- `count()`
- `min()`, `max()`, `mean()`, and `median()`
- `agg()`
- `groupby()`

info():

- ▶ **info()**: provides a concise summary of a dataframe.
- ▶ It shows you all the information you need to know about your dataframe like: record counts, column names, data types, index range , and memory usage.
- ▶ **Example:-**
- ▶ By looking at the summary provided for **ss.info()** below we can observe:
- ▶ record count is 1000
- ▶ composed of 17 columns
- ▶ Column names can be updated to eliminate white spaces
- ▶ Data types included are object, float64 and int
- ▶ Date and Time columns are dtype = object, should be updated to corresponding datetime formats.
- ▶ There are no missing values on any of the records

describe()

- ▶ Generates descriptive statistics that will provide visibility of the dispersion and shape of a dataset's distribution. It excludes NaN values. It can be used for dataframe or a specific series.
- ▶ By default, the results are for numeric types, but you can use the parameter **include** to show the stats for some or all the data types in the dataframe.
- ▶ You can exclude specific data types from a dataframe as well with the **exclude** parameter
- ▶ By default, the percentiles returned by this function are the 25th, 50th and 75th. If you wish to change the percentiles in the results, you can add those by using the parameter **percentiles**.
- ▶ Values must be between 0 and 1.

value_counts():

- ▶ This function returns counts of unique values for the specified series. NaN values are excluded by default.
- ▶ By default, the results are listed in descending order, but you can change it by using parameter ***ascending = True***
- ▶ You can get the frequency (instead of count) for each value by setting the ***normalize = True***

nunique()

- ▶ Count distinct observations. Can be used for a **dataframe** or a series. By default, it exclude the NaN values.
- ▶ **sum()**: Return the sum of the values for the requested axis. You can use it for both dataframe and series.
- ▶ You can specify to apply the function only to numeric types by using the parameter: ***numeric_only = True***
- ▶ Another useful parameter is ***min_count*** where you can set the minimum number of records that must exists to apply the function.
- ▶ **count()** result for the ss dataframe, Use the parameter ***numeric_only = True*** to apply functions to only series with numeric data types.

Min and Max, Mean and Median

- ▶ **Min and Max, Mean and Median:**
- ▶ a. **min()**: Return the minimum value
- ▶ b. **max()**: Return the maximum value
- ▶ c. **mean()**: Return the mean of the values
- ▶ d. **median()**: Return the median of the values

agg()

- ▶ **agg()**: apply more than one aggregation operations to the same dataset over the specified axis.
- ▶ Can be applied to specific series:
- ▶ Apply different aggregations to different columns:

groupby()

- ▶ **groupby()**: allows you to group data (by applying aggregate functions like sum, max, min...) with the same values into summary rows
- ▶ Apply to a specific series
- ▶ Group by more than one column
- ▶ **groupby()** function must be used with an aggregation function

